
Git e GitHub

Controle de versão e colaboração profissional



Prof. Dr. João Choma

Universidade Estadual de Maringá - Engenharia de Software

github.com/JoaoChoma

Material didático



O "Problema Exemplo" do Cotidiano

Imagine a seguinte situação prática:

- Você precisa editar um código ou trabalho que está armazenado em uma pasta compartilhada (como o Dropbox).
- Você faz o download do arquivo, faz as alterações necessárias e o salva de volta.
- **O Conflito:** Seu colega de equipe baixa o mesmo arquivo exatamente ao mesmo tempo. Ele faz outras alterações, salva no mesmo local e acaba **sobrescrevendo completamente o seu código!**

Sem uma ferramenta adequada, a colaboração em projetos de software se torna um caos de arquivos renomeados manualmente como `trabalho_final.txt`, `trabalho_final_v2_este_sim.txt` ou `trabalho_final_v2_revisado_ultima_versao.txt`.



O que é o Controle de Versão (SCM)?

Gerência de Configuração de Software

Para resolver esse problema de colaboração e perdas de alterações, foi criada a **Gerência de Configuração de Software (SCM - Source Control Management)**.

Durante o desenvolvimento de qualquer software, os desenvolvedores precisam responder a perguntas essenciais:

- **O que mudou** no código e **quando** mudou?
- **Por que** essa mudança foi realizada?
- **Quem** foi o autor da mudança?
- Nós **podemos reproduzir** essa mudança ou reverter para um estado anterior estável se algo der errado?

O sistema de controle de versão é o responsável por registrar todo o histórico de alterações em artefatos de software como código-fonte, documentação e manuais.



O Nascimento do Git

A Criação de Linus Torvalds

- O **Git** é o sistema de controle de versão mais usado no mundo atualmente.
- Ele foi criado em **2005** por **Linus Torvalds** (o mesmo criador do núcleo Linux).
- **Objetivo Inicial:** O Git foi projetado especificamente para auxiliar e coordenar o desenvolvimento descentralizado e massivo do próprio sistema operacional Linux.
- **Velocidade e Eficiência:** Comparado a sistemas anteriores como Subversion (SVN), CVS, Bazaar ou Mercurial, o Git se consolidou por ser **o mais rápido e eficiente** no mercado de TI.



Trabalho Offline e Snapshots Completos

Diferente dos sistemas centralizados antigos (onde os arquivos e históricos ficavam apenas em um servidor), o Git é **Distribuído**. Isso significa que:

- **Clone Local Completo:** Cada desenvolvedor possui uma cópia idêntica e completa de todo o histórico do projeto em sua própria máquina.
- **Trabalho Offline:** É possível realizar commits, consultar o histórico completo, criar ramificações (branches) e navegar por versões passadas **sem precisar de conexão com a internet**.
- **Snapshots:** Em vez de registrar apenas "diferenças de arquivos", o Git armazena e gerencia **snapshots** (fotos do estado de todos os arquivos) ao longo do tempo.



Preparando o Ambiente

O Git está disponível e é compatível com os principais sistemas operacionais do mercado: **Windows, macOS e Linux.**

Para instalar em sistemas baseados em Debian ou Ubuntu (como o terminal Linux de desenvolvimento), execute no terminal:

```
sudo apt-get install git
```

Interface Gráfica (GUI):

Embora o terminal (Command Line Interface - CLI) seja o método mais comum e poderoso para usar o Git, você pode optar por interfaces gráficas (GUIs) que ajudam a visualizar o repositório, como:

- **VSCode:** Possui integração nativa com o Git.
- **GitEye:** Interface multiplataforma recomendada.
- **GitKraken:** Oferece excelente visualização gráfica de conflitos e branches.



Identificando-se no Projeto

Antes de registrar sua primeira alteração, você deve configurar sua identidade. Isso é fundamental para que outros colaboradores saibam quem fez cada commit.

No seu terminal, digite os seguintes comandos substituindo pelas suas informações:

```
# Define o seu nome globalmente  
git config --global user.name "Seu Nome Completo"  
  
# Define o seu e-mail globalmente  
git config --global user.email "seu.email@exemplo.com"  
  
# Ativa cores automáticas no terminal para facilitar a leitura dos comandos  
git config --global color.ui auto
```



O Comando `git init`

Para iniciar o rastreamento de um projeto novo ou existente com o Git, usamos o comando de inicialização:

```
git init NomeDoRepositorio
```

O que acontece nos bastidores?

- 1. O Git criará uma nova pasta com o nome especificado no comando.
- 2. Dentro dessa pasta, será criado um diretório oculto especial chamado `.git`.
- 3. **A Pasta `.git`:** É o coração do seu repositório local. É nela que o Git armazena todas as configurações, o histórico completo de revisões, snapshots e metadados. **Nunca a delete manualmente**, ou você perderá todo o histórico de controle de versão do projeto!



O Ciclo de Vida dos Arquivos no Git

Os 4 Estados Essenciais

Dentro de uma pasta controlada pelo Git, os arquivos passam por um fluxo de estados bem definido:

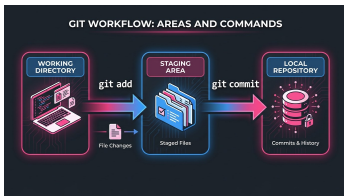
- 1. **Não monitorado (Untracked):** Arquivos novos que acabaram de ser criados na pasta, mas que o Git ainda não começou a rastrear.
- 2. **Modificado (Modified):** Arquivos já monitorados pelo Git que sofreram alterações desde o último commit, mas cujas alterações ainda não foram preparadas para salvamento.
- 3. **Preparado (Staged):** Arquivos cujas alterações modificadas foram selecionadas e preparadas para serem gravadas no próximo commit.
- 4. **Consolidado (Committed):** O estado em que os arquivos e suas alterações foram gravados de forma segura e permanente no repositório local.



Onde Ocorrem as Operações?

O Git divide conceitualmente o seu fluxo de desenvolvimento local em **três áreas de trabalho**:

- 1. **Diretório de Trabalho (Working Directory):** É a pasta física no seu computador onde você cria, edita e deleta seus arquivos diretamente.
- 2. **Área de Preparo (Staging Area / Index):** Uma área conceitual (gerenciada pelo arquivo index na pasta `.git`) onde você agrupa e revisa as alterações que deseja incluir no seu próximo registro.
- 3. **Repositório Local (.git directory):** O banco de dados onde o Git armazena permanentemente os commits e metadados do projeto.





Adicionando Alterações à Área de Staging

Quando você cria ou edita um arquivo no seu diretório de trabalho, precisa avisar ao Git que deseja incluir essas modificações no próximo "ponto de salvamento".

Para adicionar um arquivo específico à área de preparação (staging):

```
git add NomeDoArquivo
```

Para adicionar **todos** os arquivos novos e modificados da pasta atual de uma única vez, use o operador ponto:

```
git add .
```



Salvando as Alterações (O Comando `git commit`)

Gravando o Checkpoint no Histórico

Uma vez que você colocou os arquivos desejados na Área de Preparo com o `git add`, você deve consolidar essa versão de forma permanente no seu repositório local usando o comando `commit`.

```
git commit -m "Criado o primeiro arquivo de configuração do sistema"
```

Boas Práticas de Commit:

- **Descrição Clara:** A mensagem do parâmetro `-m` deve explicar claramente **o que** foi alterado ou adicionado.
- **Mensagem via Editor:** Se você digitar apenas `git commit` sem o parâmetro `-m`, o Git abrirá um editor de texto (como o `nano` ou `gedit`) para você escrever uma mensagem detalhada. Você pode configurar seu editor padrão com:

```
git config --global core.editor gedit
```



Comandos `git status` e `git log`

Para garantir o controle total das alterações, usamos frequentemente dois comandos de auditoria no terminal:

1. `git status`

Mostra o estado atual do repositório. Ele informa:

- Em qual branch você está trabalhando.
- Quais arquivos foram modificados mas não adicionados à área de staging.
- Quais arquivos estão prontos para serem commitados.
- Arquivos que não estão sendo rastreados pelo Git.

2. `git log`

Exibe a lista cronológica reversa de todos os commits realizados no repositório, detalhando:

- O código de identificação do commit (Hash SHA-1).
- O autor (Nome e E-mail).
- A data e hora exata em que o commit foi feito.
- A mensagem descritiva inserida no commit.



O que exatamente foi alterado?

Antes de fazer um commit, é fundamental revisar as linhas exatas de código que foram modificadas para evitar erros.

```
# Mostra as modificações feitas no Diretório de Trabalho que ainda não foram para staging  
git diff
```

```
# Mostra as diferenças de um arquivo ou diretório específico  
git diff [caminho_do_arquivo]
```

```
# Exibe o que foi alterado no último commit realizado em relação ao anterior  
git diff HEAD~1
```

Dica Didática:

Você pode configurar uma ferramenta externa visual de comparação de código para ajudar na resolução de diferenças e mesclagens complexas (por exemplo, o DiffMerge):

```
git config --global diff.tool diffmerge  
git difftool HEAD~1
```



Exercício Prático 1

Ciclo de Trabalho Básico Local

Para consolidar o que aprendeu, realize esta atividade no terminal:

- 1. Crie um arquivo chamado README.md usando seu editor de texto.
- 2. Digite as seguintes linhas no terminal para verificar o estado e registrar o arquivo:

```
git status
git add README.md
git commit -m "Adiciona o arquivo README inicial do projeto"
```

- 3. Abra o arquivo, modifique uma das linhas e adicione uma nova frase.
- 4. Verifique as alterações no código antes de salvar:

```
git diff
```

- 5. Consolide a nova versão:

```
git add .
git commit -m "Atualiza informações do README"
git log
```



Viagem no Tempo no seu Código

Uma das maiores vantagens do Git é a capacidade de reverter ou inspecionar o estado de um arquivo no passado.

Se você quiser ver como um arquivo estava em um determinado commit anterior:

```
git checkout <hash_do_commit> <caminho_do_arquivo>
```

Se quiser colocar temporariamente **todo o repositório** no estado de um commit do passado para rodar testes:

```
git checkout <hash_do_commit>
```

Para retornar ao estado atual do desenvolvimento na linha de código padrão (master/main):

```
git checkout master
```




O Uso Seguro de reset e revert

O Git oferece comandos específicos para desfazer alterações locais ou públicas de forma controlada.

1. git reset (Remover Alterações)

Este comando altera o histórico do repositório para o estado de um commit específico do passado.

```
git reset --hard <hash_do_commit>
```

Cuidado Extremo: O parâmetro `--hard` remove e apaga todas as alterações locais feitas após o commit especificado. **Não utilize este comando se o seu código já tiver sido compartilhado publicamente** com outros colegas no servidor remoto!

2. git revert (Desfazer Amigavelmente)

Diferente do reset, o revert cria um **novo commit de correção** que realiza a operação inversa do commit indesejado, preservando o histórico público intacto.



O Conceito de Linhas Paralelas

- **O que é um Branch?** É uma linha de desenvolvimento separada no seu repositório. No fundo, um branch é apenas um ponteiro móvel que aponta para uma lista sequencial de commits.
- **Isolamento de Funcionalidades:** Quando você precisa desenvolver uma funcionalidade complexa ou corrigir um bug (hotfix), cria um branch separado para que as alterações em andamento **não quebrem o código estável** principal.
- **Branch Padrão:** O branch padrão criado automaticamente quando inicializamos o repositório é o **master** ou **main**.

```
# Lista todas as branches locais existentes no projeto  
git branch
```



Criar, Alternar e Excluir Branches

O gerenciamento de ramificações no Git é extremamente leve e rápido:

```
# 1. Cria uma nova linha de desenvolvimento separada
git branch feature-novidade

# 2. Muda o ambiente de trabalho ativo para a branch criada
git checkout feature-novidade

# Alternativa rápida: Criar e mudar para a branch em um único comando
git checkout -b feature-novidade

# 3. Exclui um branch após concluir o desenvolvimento e mesclagem
git branch -d feature-novidade
```

Uma vez que você altera para o branch feature-novidade, os novos commits serão anexados exclusivamente a ele, permitindo que a linha master principal continue intacta.



Integrando Branches: O Comando `git merge`

Unindo Linhas de Código

Após desenvolver e testar com sucesso uma nova funcionalidade no seu branch separado, você desejará unir essas alterações de volta ao branch principal (`master`).

Para realizar a fusão:

- 1. Posicione-se no branch que vai receber o código (neste caso, o `master`):

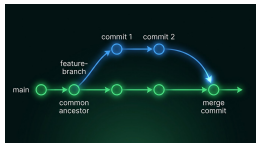
```
git checkout master
```

- 2. Execute o comando de fusão especificando o branch de origem:

```
git merge feature-novidade
```

Como o merge funciona?

O Git procura o ancestral comum (a base) entre as duas branches e aplica automaticamente as modificações feitas na branch secundária em cima da branch principal. Se não houver divergências conflitantes, a fusão é concluída com sucesso.





Mantendo um Histórico Linear e Limpo

O comando `git rebase` é uma alternativa elegante ao `git merge`.

```
git rebase master
```

Diferença entre Merge e Rebase:

- **No merge:** O Git une as ramificações criando um novo commit histórico especial de mesclagem (commit de merge).
- **No rebase:** O Git reescreve o histórico temporariamente. Ele remove temporariamente seus commits feitos em paralelo, atualiza seu branch com os commits recentes do master e, por fim, aplica seus commits um a um no final da fila.
- **Vantagem:** O histórico de commits se torna totalmente **linear**, facilitando a leitura e auditoria das versões do software.



O que fazer quando o Git trava?

- **O que gera um Conflito?** Conflitos acontecem quando duas versões diferentes alteram **as mesmas linhas do mesmo arquivo** de formas distintas.
- **Detecção Automática:** O Git não consegue decidir sozinho qual alteração deve prevalecer. Ele interrompe a operação de mesclagem (merge ou rebase) e insere marcações visuais nos arquivos conflitantes, aguardando que o desenvolvedor decida.
- **Resolução:**
 1. Abra o arquivo com conflito no seu editor.
 2. Remova as marcações do Git e decida qual código manter (ou mescle ambos).
 3. Adicione o arquivo resolvido e realize o commit para finalizar:

```
git add arquivo_conflito.js  
git commit -m "Resolve conflito de mesclagem no arquivo de script"
```



A Rede Social do Código-Fonte

- **Servidor Remoto:** O GitHub (surgido em 2008) é uma plataforma online de hospedagem e compartilhamento de repositórios Git na nuvem.
- **Portfolio Profissional:** Atualmente, o perfil do GitHub é altamente valorizado por empresas de tecnologia em processos seletivos de recrutamento e contratação de desenvolvedores.
- **Gigantes do Setor:** Grandes corporações globais e órgãos governamentais (como Google, NASA, Microsoft, Netflix, Facebook e Spotify) hospedam e mantêm seus projetos de código aberto ativamente no GitHub.



Sincronizando com o GitHub (Remotes)

Conectando Local e Nuvem

Para hospedar seu projeto no GitHub, você precisa adicionar o endereço do repositório remoto ao seu repositório local.

```
# Cria um vínculo local com o repositório remoto apontando para uma URL  
git remote add origin https://github.com/SeuUsuario/SeuRepositorio.git
```

O termo **origin** é o nome padrão convencional adotado para representar o seu servidor remoto principal no GitHub.



Publicando seu Trabalho na Nuvem

Uma vez configurado o repositório remoto, você pode enviar os seus commits locais para o servidor do GitHub.

Para enviar seus commits pela primeira vez e definir o rastreamento padrão:

```
git push -u origin master
```

Nas próximas vezes, basta executar o comando de forma simplificada:

```
git push
```

Nota: O envio será rejeitado pelo servidor caso o seu repositório local não esteja sincronizado com as últimas atualizações públicas feitas por outros colegas.



Mantendo seu Código em Sincronia

Quando trabalha em equipe, você deve baixar com frequência as atualizações feitas pelos seus colegas no servidor remoto.

Temos duas estratégias principais para sincronizar seu repositório local:

1. `git fetch`

Baixa as atualizações do servidor remoto para o seu computador, mas **não as aplica** ou mescla nos seus arquivos locais de trabalho. Permite analisar o código remoto de forma segura primeiro.

2. `git pull`

Baixa as atualizações e **realiza a mesclagem automática** no seu branch ativo imediatamente.

- **Fórmula do Pull:** `git pull` equivale exatamente a executar um `git fetch` seguido de um `git merge`.



Baixando Código da Comunidade

Se você deseja obter e colaborar em um repositório que já existe no GitHub (seu ou de outro desenvolvedor), você usa o comando `clone`.

```
git clone https://github.com/Usuario/NomeDoProjeto.git
```

Vantagens do `git clone`:

- **Completo:** Baixa automaticamente todo o histórico de alterações do projeto.
- **Configuração Pronta:** O repositório já é baixado localmente com a conexão com o servidor remoto (origin) configurada de forma automática.



Sua Cópia Independente na Nuvem

- **O que é um Fork?** É uma bifurcação. O Fork copia todo um repositório público de outra pessoa para a **sua própria conta pessoal** do GitHub.
- **Liberdade de Alteração:** Com a cópia na sua conta pessoal, você tem total liberdade para realizar clonagens, alterar código, commitar e fazer testes sem risco de afetar ou danificar o projeto original alheio.
- **O Início da Colaboração:** É assim que se inicia a contribuição prática para grandes projetos de código aberto (Open Source) na internet.



Propondo Alterações com Pull Request (PR)

O Símbolo Máximo da Colaboração

- **O que é o Pull Request?** É o mecanismo formal pelo qual você solicita que as alterações feitas no seu repositório bifurcado (fork) ou em um branch separado de um repositório compartilhado sejam **unidas (mergeadas)** ao branch do projeto principal.
- **Revisão e Discussão:** O Pull Request abre uma interface gráfica no GitHub que permite aos mantenedores do projeto revisar detalhadamente suas linhas de alteração de código, sugerir melhorias e discutir a implementação antes de incorporá-la.





Organizando as Tarefas do Time

O GitHub oferece ferramentas integradas de gestão que facilitam o desenvolvimento colaborativo:

- **Issues (Questões):** É uma seção no repositório que serve para documentar e organizar discussões, dúvidas e propostas de funcionalidades.
- **Rastreamento de Bugs:** É o local oficial para relatar bugs encontrados nos programas.
- **Associação com Commits:** Você pode vincular um commit ou pull request diretamente a uma Issue. Por exemplo, ao incluir o termo "Closes #3" na mensagem de um commit, o GitHub fechará automaticamente a Issue de número 3 quando o commit for mesclado.



Ferramentas e Recursos de Apoio

Para maximizar sua produtividade, explore estes recursos avançados suportados pelo Git:

- **.gitignore:** Um arquivo de configuração especial onde você lista pastas e extensões de arquivos (como arquivos temporários, logs ou chaves secretas) que o Git deve ignorar e nunca rastrear.
- **git stash:** Guarda temporariamente suas alterações locais inacabadas "na prateleira" para que você possa limpar seu ambiente de trabalho, mudar de branch para resolver uma emergência e, depois, recuperar seu trabalho de onde parou.
- **git cherry-pick:** Permite selecionar e aplicar um commit específico de um branch diretamente em outro branch.
- **Documentação Oficial:** Acesse o site oficial do Git: git-scm.com



Dúvidas e Considerações Finais

O Sucesso está na Prática!

O domínio do Git e do GitHub não é puramente conceitual — ele exige **prática diária e constante** de programação e colaboração em equipes.

Resumo do Fluxo Principal de Trabalho (Workflow):

```
# 1. Atualize o seu código local
git pull

# 2. Crie sua branch de desenvolvimento
git checkout -b feature-minha-tarefa

# 3. Edite o código e prepare os arquivos
git add .

# 4. Registre suas modificações localmente
git commit -m "Implementa a nova funcionalidade XYZ"

# 5. Envie suas alterações para o GitHub
git push origin feature-minha-tarefa

# 6. Abra um Pull Request para revisão da equipe!
```

Obrigado pela atenção!

Dúvidas e discussões abertas a qualquer momento.